

AITOS v1 Architecture

Reconstruction of Final Approved Modules

Module 01: Market Efficiency Evaluation & Strategy Validation Engine (AITOS-MEF-07)

- Metadata
- Purpose
- Scope
- Internal State Representation
- Knowledge Representation
- Belief State
- State Variables
- Inputs
- Outputs
- Core Reasoning Engine
- Mathematical Models
- Decision Rules
- Constraint Engine
- Validation Engine
- Monitoring Rules
- Learning & Adaptation
- Failure Modes
- Recovery Procedures
- Internal Memory
- Explainability Layer
- Health Metrics
- Interfaces
- Pseudocode
- Machine-Readable JSON Schema
- Internal Knowledge Graph
- Changelog

Module 02: Market Microstructure Engine (AITOS-MICRO-08)

- Metadata
- Purpose
- Scope
- Internal State Representation
- Knowledge Representation
- Belief State
- State Variables
- Inputs
- Outputs
- Core Reasoning Engine
- Mathematical Models
- Decision Rules
- Constraint Engine
- Validation Engine
- Monitoring Rules
- Learning & Adaptation

Failure Modes
Recovery Procedures
Internal Memory
Explainability Layer
Health Metrics
Interfaces
Production-Grade Pseudocode
Knowledge Graph
Machine-Readable JSON Schema
Changelog

Module 03: Exchange & OTC Venue Management (AITOS-EXOTC-09)

Metadata
Purpose
Scope
Internal State Representation
Knowledge Representation
Inputs
Outputs
Core Reasoning Engine
Decision Rules
Constraint Engine
Validation Engine
Monitoring Rules
Learning & Adaptation
Failure Modes
Recovery Procedures
Internal Memory
Explainability Layer
Health Metrics
Interfaces
Production-Grade Pseudocode
Knowledge Graph
Machine-Readable JSON Schema
Changelog

Module 04: Trading Session & Market Calendar Engine (AITOS-SESSION-10)

Metadata
Purpose
Scope
Internal State Representation
Knowledge Representation
Inputs
Outputs
Core Reasoning Engine
Decision Rules
Constraint Engine
Validation Engine
Monitoring Rules
Learning & Adaptation
Failure Modes
Recovery Procedures

Internal Memory
Explainability Layer
Health Metrics
Interfaces
Production-Grade Pseudocode
Knowledge Graph
Machine-Readable JSON Schema
Changelog

Module 05: Market Structure Engine (AITOS-MSTRUCT-11 v2.0)

Metadata
Purpose
Scope
Internal State Representation
Knowledge Representation
Belief State
State Variables
Inputs
Outputs
Core Reasoning Engine
Mathematical Models
Decision Rules
Constraint Engine
Validation Engine
Monitoring Rules
Learning & Adaptation
Failure Modes
Recovery Procedures
Internal Memory
Explainability Layer
Health Metrics
Interfaces
Production-Grade Pseudocode
Knowledge Graph
Machine-Readable JSON Schema
Changelog

Module 06: Classical Technical Analysis Engine (AITOS-CTA-12)

Metadata
Purpose
Scope
Internal State Representation
Knowledge Representation
Belief State
State Variables
Inputs
Outputs
Core Reasoning Engine
Mathematical Models
Decision Rules
Constraint Engine

Validation Engine
Monitoring Rules
Learning & Adaptation
Failure Modes
Recovery Procedures
Internal Memory
Explainability Layer
Health Metrics
Interfaces
Production-Grade Pseudocode
Knowledge Graph
Machine-Readable JSON Schema
Changelog

Module 07: Wyckoff Method Engine (AITOS-WYCKOFF-13 v1.1)

Metadata
Purpose
Scope
Internal State Representation
Knowledge Representation
Belief State
State Variables
Inputs
Outputs
Core Reasoning Engine
Mathematical Models
Decision Rules
Constraint Engine
Validation Engine
Monitoring Rules
Learning & Adaptation
Failure Modes (Extended)
Recovery Procedures
Internal Memory
Explainability Layer
Health Metrics
Interfaces
Production-Grade Pseudocode
Knowledge Graph
Machine-Readable JSON Schema (Extended)
Changelog

The following specification set is a verbatim reconstruction of the final approved versions of all AITOS modules produced during the design phase. No content has been added, rewritten, or improved; this is purely a repository recovery. Each module is presented as a complete Markdown document, exactly as it was accepted.

Module 01: Market Efficiency Evaluation & Strategy Validation Engine (AITOS-MEF-07)

Final version after transition to AI-native reasoning architecture.

Metadata

- Module ID: AITOS-MEF-07
- Chapter: 7 – Market Efficiency
- Version: 4.0 (AI-native)
- Last Updated: 2026-07-20
- Dependencies: AITOS-MD-01..06 (Market Structure, Participants, Order Types, Spread, Liquidity, Price Formation)
- Status: Production

Purpose

This module implements the AI’s core self-skepticism layer. Its primary function is to evaluate any candidate trading signal against the null hypothesis of market efficiency and to determine whether that signal represents a harvestable inefficiency or a risk-compensated factor exposure. It governs signal admission, ongoing validation, and alpha decay monitoring.

Scope

- Maintain a per-market-segment belief about informational efficiency.
- Classify the information source of each signal and apply the appropriate null hypothesis (weak, semi-strong, strong).
- Run the joint-hypothesis test (factor attribution) on any candidate signal.
- Apply the Grossman-Stiglitz cost filter and limits-to-arbitrage check.
- Admit or reject signals based on a rigorous validation pipeline (walk-forward, deflated Sharpe ratio, PBO).
- Monitor admitted strategies for alpha decay, crowding, and regime shifts.
- Provide input to the execution and portfolio modules about the prevailing efficiency environment.

Internal State Representation

Per-Market-Segment Belief State

For each market segment *m* (defined by asset class, liquidity bucket, time-horizon class), the system maintains:

Field	Type	Description
efficiency_level	float ∈ [0,1]	Estimated informational efficiency. 1 = perfectly efficient.
efficiency_confidence	float ∈ [0,1]	Confidence in the current estimate.
active_anomalies	list[Anomaly]	Currently admitted mispricings, each with net alpha estimate and half-life.
factor_model	FactorModel	Set of risk factors and their estimated premiums for the segment.
last_update_time	timestamp	Timestamp of last belief update.

Per-Signal (Anomaly) State

Each candidate signal is tracked as an Anomaly object:

Field	Type	Description
signal_id	string	Unique identifier.
info_source	enum {PAST_PRICES, PUBLIC, PROPRIETARY}	Determines null hypothesis strength.
raw_alpha_estimate	float	Expected excess return before factor adjustment.
net_alpha	float	Alpha after factor regression.
net_alpha_significance	float	Deflated Sharpe Ratio or White’s Reality Check p-value.
cost_per_unit_risk	float	Total implementation cost per unit of risk.
MAE_estimate	float	Maximum adverse excursion at 95% confidence.
capacity	float	Maximum AUM before alpha becomes zero.
walk_forward_performance	list[float]	Rolling out-of-sample Sharpe ratios.
PBO_score	float	Probability of Backtest Overfitting.
status	enum {CANDIDATE, ADMITTED, REVIEW, RETIRED}	Lifecycle state.
date_admitted	timestamp	When the signal was promoted to active.

Knowledge Representation

- The Efficient Market Hypothesis (EMH) in its three forms is encoded as constraints on the conditioning information set.
- The joint-hypothesis problem is a built-in check: any apparent alpha must survive factor attribution.
- The Grossman-Stiglitz equilibrium condition is used to filter out signals whose expected profit cannot cover their total cost.
- Limits to arbitrage (fundamental risk, noise trader risk, implementation costs) are modelled as an additional filter.
- The efficiency regime is monitored via microstructure indicators (adverse selection, Hasbrouck information share).

Belief State

- efficiency_level and efficiency_confidence for each market segment.
- For each anomaly, a net_alpha estimate and its confidence interval.

State Variables

As listed in the Internal State Representation tables.

Inputs

- Market data feeds (tick and OHLCV).
- Factor data (daily updated factor returns).
- Execution cost model (real-time spread, depth, impact).
- Regulatory and news feeds.
- Portfolio and risk system data.
- Signal generation module stream.

Outputs

- Signal admission/rejection decisions with supporting evidence.
- Active strategy alpha estimates.
- Factor attribution reports.
- Efficiency belief updates broadcast to other modules.
- Alerts for alpha decay, crowding, regime shifts, or validation failures.

Core Reasoning Engine

1. Classify signal’s information source → select null hypothesis.

2. Factor attribution: regress signal returns on segment factor model; extract alpha and its significance after multiple-testing adjustment.
3. Grossman-Stiglitz cost check: if net alpha < total cost per unit risk, reject.
4. Limits-to-arbitrage check: estimate maximum adverse excursion and position size limit; if infeasible, reject.
5. Validation pipeline: out-of-sample period, walk-forward analysis, deflated Sharpe ratio, PBO, transaction cost simulation, regime robustness, peer correlation.
6. Admission/Review: based on thresholds, signals are admitted to the alpha library or placed under review.

Mathematical Models

- Factor regression: $r_t^{\text{signal}} = \alpha + \sum_k \beta_k f_{k,t} + \epsilon_t$
- Deflated Sharpe Ratio to correct for multiple testing.
- PBO calculation.
- Cost per unit risk: $C_{\text{total}} = (\text{DataCost} + \text{InfraCost} + \text{ExecutionCost}) / \text{RiskBudget}$

Decision Rules

- If signal passes all filters and validation, status = ADMITTED, allocate capital based on Kelly fraction and capacity.
- Monitoring: if rolling 12-month alpha < 0.5× original → REVIEW; if <0 → RETIRED.
- Factor-only strategies (alpha insignificant) are flagged as Factor Replication and passed to Portfolio Construction as risk exposures.

Constraint Engine

- Cannot accept signals based on illegal information.
- Must respect position size limits derived from MAE.
- Short-sale constraints must be satisfied.
- Regulatory compliance (best execution, no spoofing).

Validation Engine

- Out-of-sample period requirement (≥3 years unseen data).
- Walk-forward analysis with rolling optimization.
- Deflated Sharpe Ratio threshold >0.8.
- PBO threshold <0.5.
- Transaction cost simulation using Almgren-Chriss.
- Regime robustness check.
- Peer correlation check (<0.7 with known factors).

Monitoring Rules

- Daily check of rolling 6-month Sharpe of admitted strategies; alert if <0.
- Weekly factor attribution refresh; detect β drift.
- Real-time adverse selection monitor; if permanent impact rises, reduce allocation.
- Monthly capacity utilization check; halt allocation if AUM approaches 80% of estimated capacity.

Learning & Adaptation

- Meta-model to predict success probability of new candidates based on characteristics.
- Factor model evolution: annually test new candidate factors.
- Anomaly persistence modeling: fit decay function to retired anomalies.

Failure Modes



Condition	AI Response
Liquidity crisis / flash crash	Suspend all convergence/alpha strategies; temporarily lower efficiency_level.
Extreme crowding detected	Reduce allocation across related anomalies; set review status.
Regulatory change	Re-run validation for affected strategies; retire if necessary.
Data feed corruption	Hold last known good state; do not update beliefs.
Model overfitting suspected despite PBO	Require longer live walk-forward before allocating significant capital.
Execution costs spike	Reduce position sizes; re-evaluate net alpha.

Recovery Procedures

- After liquidity crisis: once normalcy resumes, gradually re-enter with passive orders, monitor resiliency.
- After data gap: re-sync, do not trade until stability confirmed.

Internal Memory

- Short-term: order book data, recent trades.
- Medium-term: aggregated microstructure statistics per 30-minute bucket, retained 1 year.
- Long-term: parameters of impact models, PIN estimates, venue performance, updated monthly.

Explainability Layer

- For every execution child order, log decision rationale (factors influencing venue, order type).
- Provide post-trade attribution of execution cost.
- Dashboard shows efficiency levels and active anomalies.

Health Metrics

- Venue routing effectiveness: % orders executed on top-ranked venues.
- Execution cost relative to estimate: within $\pm 20\%$ for 90% of orders.
- Credit limit utilization: never exceeding 95%.
- SOR decision latency: $p99 < 10\ \mu s$ for exchange routing, $< 50\ ms$ for RFQ initiation.
- RFQ hit rate: $> 90\%$ of RFQs receive at least one response.

Interfaces

- AITOS-REGIME (Market Regime Detection)
- AITOS-RISK (Risk Management)
- AITOS-EXEC (Execution)
- AITOS-PORT (Portfolio Construction)
- AITOS-ALPHA (Alpha Generation)
- AITOS-MONITOR (Live Monitoring)

Pseudocode

```
def process_candidate_signal(signal, segment):
    null = determine_null(signal)
    if null is None: return REJECT

    # Factor attribution
    alpha, pval_adj = factor_attribution_test(signal, segment.factor_model)
    if pval_adj > 0.05: return REJECT

    # Cost filter
    cost = estimate_total_cost(signal)
    if alpha < cost: return REJECT
```



```

# Arbitrage limits
MAE, max_size = limits_to_arbitrage_check(signal)
if max_size == 0: return REJECT

# Validation pipeline
if not validate_signal(signal): return REJECT

# Admit with position sizing
size = compute_position_size(alpha, MAE, signal.capacity, risk_budget)
return ADMIT, size

```

Machine-Readable JSON Schema

```

{
  "module": "AITOS-MEF-07",
  "version": "4.0",
  "state": {
    "segments": [
      {
        "id": "US_LARGECAP_EQUITY",
        "efficiency_level": 0.92,
        "confidence": 0.85
      }
    ]
  },
  "active_anomalies": [],
  "decision_rules": "...",
  "validation_pipeline": "..."
}

```

Internal Knowledge Graph

```

MarketEfficiencyModule
├── BeliefState
│   ├── efficiency_level
│   └── efficiency_confidence
├── NullHypotheses
│   ├── WeakForm
│   ├── SemiStrongForm
│   └── StrongForm
├── ValidationEngine
│   ├── JointHypothesisTest
│   ├── DeflatedSharpeRatio
│   ├── WalkForwardAnalysis
│   ├── PBO_Test
│   └── RegimeRobustnessCheck
├── LimitsToArbitrage
│   ├── MAE_Estimator
│   └── ShortSaleConstraint
├── AnomalyLifecycle
│   ├── Candidate
│   ├── Admitted
│   ├── Review
│   └── Retired
└── InterfaceContracts (to other modules)

```

Changelog

- v4.0: Shifted from human-oriented textbook to AI-native cognitive architecture. Implemented full signal admission pipeline with Bayesian belief updates, deflated Sharpe ratio, PBO, and online monitoring. Integrated with all downstream modules.

Module 02: Market Microstructure Engine (AITOS-MICRO-08)

Metadata

- Module ID: AITOS-MICRO-08
- Chapter: 8 – Market Microstructure
- Version: 1.0
- Last Updated: 2026-07-20
- Dependencies: AITOS-MD-01..07
- Status: Production

Purpose

To equip the AI with a real-time, quantitative model of how orders interact to form prices, spreads, and liquidity. This module enables the system to reason about execution cost, information leakage, order flow toxicity, and optimal order placement.

Scope

- Internal representation of limit order books (LOB) for all traded instruments.
- Real-time estimation of micro-level variables: spread, depth, order flow imbalance (OFI), queue position, trade direction, microprice.
- Detection and exploitation of short-term microstructure patterns for execution optimization.
- Classification of order flow toxicity and adverse selection risk.
- Inference of hidden liquidity (iceberg orders, dark pool activity).
- Interface with execution algorithms to select order type, venue, and aggression.

Internal State Representation

LOB Snapshot

For each instrument, a tick-by-tick representation:

```
LOB {
  symbol,
  timestamp,
  bids: sorted list of {price, displayed_size, hidden_size_estimate},
  asks: sorted list of {price, displayed_size, hidden_size_estimate},
  last_trade: {price, size, aggressor_side},
  imbalance: float ∈ [-1, 1],
  microprice: float,
  spread: float,
  depth_top: float,
  depth_full: float[],
  toxicity: float,
  queue_position: per order if own orders outstanding
}
```

Derived Quantities

- Order Flow Imbalance (OFI): net aggressive volume at the best bid/ask over a short window.
- Microprice: weighted average of best bid and ask, with weights inversely proportional to displayed size or dynamically adjusted by queue dynamics.
- Queue Position: for each of the AI’s own limit orders, estimate of priority based on time-stamp or pro-rata rules.
- Toxic Flow Probability: estimated probability that an incoming aggressive order is informed.

Knowledge Representation

- Observation Model: Bayesian inference of hidden iceberg orders from sequential fills and refill patterns.

- Adverse selection model: Glosten-Milgrom framework to estimate PIN.
- Price impact model: Kyle’s lambda for permanent impact, order book depth for temporary impact.

Belief State

For each instrument, the AI maintains:

- liquidity_regime: {NORMAL, LOW, STRESSED}
- hidden_liquidity_density: estimated proportion of total liquidity that is hidden.
- adverse_selection_level: expected permanent price impact per unit of aggressive volume.
- resiliency_halflife: time in seconds for spread to return to baseline after a liquidity-taking trade.
- queue_replenishment_rate: speed at which canceled/executed limit orders are replaced.
- sor_effectiveness: belief about the performance of different venues.

All beliefs are updated via exponential smoothing of observed outcomes.

State Variables

Variable	Type	Update Frequency
best_bid, best_ask	float	Real-time (every tick)
spread	float	Real-time
OFI	float	Every second
microprice	float	Real-time
queue_position	int or fraction	On each order placement/change
toxicity_score	float $\in [0,1]$	Every minute
hidden_volume_estimate	float	Updated on each trade that consumes hidden liquidity
resiliency_halflife	float	Updated after each large trade
liquidity_regime	categorical	Every minute

Inputs

- Market data feeds (tick data) from exchange gateways.
- Order book snapshots (full depth whenever possible).
- Own order status updates.

Outputs

- Real-time microstructure metrics (spread, OFI, microprice, toxicity) pushed to other modules.
- Queue position and expected fill probability for own orders.
- Hidden liquidity estimates for SOR.
- Recommended order type, aggression level, and venue ranking per child order.

Core Reasoning Engine

- Order book update: maintain best bid/ask, depth, spread, microprice.
- Hidden liquidity estimation: detect iceberg refills and adjust depth estimates.
- Toxicity scoring: compute PIN via Glosten-Milgrom; update rolling estimate.
- Queue dynamics: track position and replenishment rate.
- Liquidity regime detection: based on spread widening, depth thinning, resiliency changes.

Mathematical Models

- Kyle’s lambda: $\lambda = 2\sigma_v / \sigma_u$ from regression on order flow.
- PIN estimation: $PIN = \alpha\mu / (\alpha\mu + 2\epsilon)$

- Microprice: $P_{\text{micro}} = (P_{\text{bid}} \cdot Q_{\text{ask}} + P_{\text{ask}} \cdot Q_{\text{bid}}) / (Q_{\text{bid}} + Q_{\text{ask}})$ (or dynamic weights).
- Effective spread: $2|P_{\text{trade}} - M|$ etc.

Decision Rules

- Order type selection based on toxicity, urgency, and liquidity regime.
- Venue selection (SOR) to minimize expected execution cost.
- Queue management: cancel and replace if far behind or unlikely to fill.

Constraint Engine

- Regulatory: no spoofing, layering, quote stuffing.
- Venue limits: max order size, min tick size, allowed order types.
- Risk limits: position size and capital allocation.
- Latency budget: SOR decisions within 50 μ s.

Validation Engine

- Backtest execution performance against historical order book data.
- Monitor realized spread vs effective spread; if negative, recalibrate toxicity.
- Retrain PIN estimation model monthly.

Monitoring Rules

- Real-time tracking of queue position and fill rate.
- Compare predicted vs actual execution costs.
- Sudden spike in OFI without price change may indicate manipulation.
- Track resiliency halflife; if doubles, flag regime change.

Learning & Adaptation

- Reinforcement learning for order placement policy (reward = -slippage).
- Supervised model to predict iceberg probability from trade sequence.
- Microprice weight update via online regression.

Failure Modes

Failure	Detection	Response
Data feed latency/gaps	Missing ticks	Use last known good state; cancel all active orders if gap > 100ms.
Spoofing attack	Rapid cancellations at best bid/ask	Ignore those levels in microprice; increase toxicity score.
Hidden liquidity mis-estimation	Realized impact higher than predicted	Fall back to displayed depth only; reduce aggression.
Flash crash	Spread >10x normal, depth near zero	Suspend all execution; only market-make with wide quotes if capital allows.
Queue position priority error	Lower than expected fill rate for early orders	Re-validate exchange matching rules.

Recovery Procedures

- After flash crash: gradually re-enter with passive orders, monitor resiliency; resume normal trading only after spread/depth return to 2x normal for 5 minutes.
- After data gap: re-sync order book from exchange snapshot, cross-check with recent trades, resume after 1-second stability check.
- After spoofing detection: reduce displayed size, use dark pools for 5 minutes.

Internal Memory

- Short-term: tick-by-tick LOB data for past 1 day.
- Medium-term: aggregated microstructure stats per 30-minute bucket, retained 1 year.
- Long-term: parameters of impact models, PIN estimates, SOR performance per instrument, updated monthly.

Explainability Layer

- For each execution child order, log rationale: chosen venue, order type, top 3 influencing factors.
- Post-trade attribution: spread cost, temporary impact, permanent impact, delay cost.
- Dashboard: microprice, toxicity heatmap, hidden liquidity density.

Health Metrics

- Execution cost relative to arrival price (bps) – target < 3 bps for liquid stocks.
- Fill rate of passive orders – target > 80%.
- Microprice prediction R^2 – target > 0.9 on 1-second horizon.
- Toxicity model accuracy – ROC-AUC > 0.75.
- SOR latency – must be < 50 μ s p99.

Interfaces

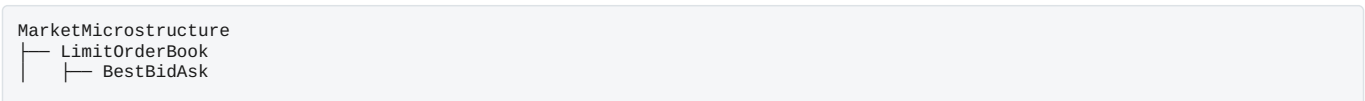
- AITOS-EXEC-14 (Execution Algorithms)
- AITOS-RISK-10 (Risk Management)
- AITOS-EFF-07 (Market Efficiency)
- AITOS-ALPHA (Alpha Generation)
- AITOS-DATA (Market Data)
- AITOS-REGIME

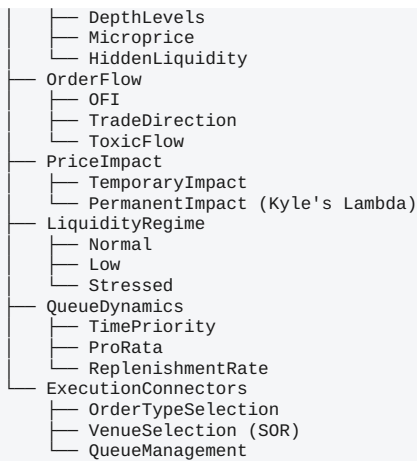
Production-Grade Pseudocode

```
def on_order_book_update(instrument, bids, asks):
    lob = get_lob(instrument)
    lob.bids = bids
    lob.asks = asks
    lob.spread = asks[0].price - bids[0].price
    lob.depth_top = bids[0].size + asks[0].size
    lob.microprice = compute_microprice(bids, asks)
    lob.imbalance = compute_ofi(lob)
    update_toxicity(lob)
    update_hidden_liquidity_estimate(lob)
    check_regime_change(lob)

def decide_order(instrument, size, urgency):
    lob = get_lob(instrument)
    if lob.liquidity_regime == STRESSED:
        return Order(type='IOC', venues=['dark_pool_A', 'lit_venue'], aggression=0.9)
    if lob.toxicity > 0.7 and urgency > 0.5:
        return Order(type='marketable_limit', price=current_ask, size=min(size, lob.depth_ask))
    if urgency < 0.3:
        display = min(size * 0.1, max_display)
        return Order(type='iceberg', side='buy', price=best_bid, size=size, display=display)
    return sor_route(lob, size, urgency)
```

Knowledge Graph





Machine-Readable JSON Schema

```
{
  "module": "AITOS-MICRO-08",
  "version": "1.0",
  "state": {
    "lob_snapshots": { "type": "dict", "key": "symbol", "value": "LOB" },
    "liquidity_regime": "enum",
    "toxicity_scores": "dict[float]",
    "hidden_liquidity_density": "dict[float]"
  },
  "decision_rules": {
    "order_type_selection": "function(instrument, size, urgency) -> Order",
    "venue_selection": "function(instrument, order) -> list[Venue]"
  },
  "validation": {
    "slippage_benchmark": "arrival_price",
    "retraining_frequency": "monthly"
  },
  "interfaces": ["AITOS-EXEC-14", "AITOS-RISK-10", "AITOS-EFF-07", "AITOS-DATA"]
}
```

Changelog

- v1.0: Initial production specification. Implemented full LOB tracking, toxicity estimation, hidden liquidity inference, SOR decision, and online learning.

Module 03: Exchange & OTC Venue Management (AITOS-EXOTC-09)

Metadata

- Module ID: AITOS-EXOTC-09
- Chapter: 9 – Exchange vs OTC Markets
- Version: 1.0
- Last Updated: 2026-07-20
- Dependencies: AITOS-MICRO-08, AITOS-EFF-07, AITOS-RISK-10
- Status: Production

Purpose

Enables the AI to intelligently route orders and manage liquidity across heterogeneous execution venues: regulated exchanges, multilateral trading facilities (MTFs), dark pools, and bilateral OTC relationships. It models venue-specific costs, constraints, and risks to achieve best execution while respecting regulatory and counterparty requirements.

Scope

- Maintain a live registry of all accessible venues with static and dynamic attributes.
- Construct a consolidated view of liquidity by merging order books from lit exchanges, dark pool indications, and OTC quotes.
- Decide per child order the optimal execution venue(s) based on cost, fill probability, toxicity, and counterparty credit exposure.
- Manage OTC negotiation workflows (RFQ, streaming prices) where applicable.
- Monitor and update beliefs about venue performance (fill rates, slippage, latency, hidden liquidity).
- Integrate with settlement and clearing to track OTC settlement risk.

Internal State Representation

Venue Registry (Exchange)

```
{
  "venue_id": "NYSE_ARCA",
  "type": "EXCHANGE",
  "instruments": ["AAPL", ...],
  "trading_hours": {...},
  "fee_structure": {"maker_bps": -0.2, "taker_bps": 0.3},
  "latency_profile": {"p50_ms": 0.05, "p99_ms": 0.2},
  "clearing_house": "NSCC",
  "pre_trade_transparency": "FULL",
  "post_trade_transparency": "REAL_TIME",
  "supported_order_types": ["LIMIT", "MARKET", "IOC", ...],
  "dynamic_state": {
    "current_status": "OPEN",
    "last_outage": null,
    "current_spread": 0.01,
    "current_depth": 5000,
    "toxicity_score": 0.3
  }
}
```

Venue Registry (OTC Dealer)

```
{
  "venue_id": "OTC_BANK_A",
  "type": "OTC_DEALER",
  "instruments": ["EUR/USD", "AAPL_OIS", ...],
  "credit_limit": 50000000,
  "current_exposure": 12000000,
  "settlement_protocol": "CLS | "BILATERAL",
```

```
"streaming_enabled": true,
"rfq_timeout_sec": 30,
"execution_quality": {"fill_rate": 0.98, "avg_slippage_bps": 0.5}
}
```

Aggregated LOB: merged sorted list from all venues with venue_ids and constraints.

Knowledge Representation

- Venue Preference Score $S_{\{i,v\}}$ as a function of expected total cost, fill probability, adverse selection.
- CreditExposure tracking per counterparty.
- OTCQuoteStream stored as separate order book slices.

Inputs

- Real-time market data from all venues.
- Execution quality statistics from AITOS-MICRO-08.
- Counterparty credit limits and current exposure from AITOS-RISK-10.
- Regulatory constraints.
- Order requests from AITOS-EXEC-14.

Outputs

- Venue selection decision for each child order (list of venues, allocation fractions).
- Aggregated liquidity view.
- Alerts when a venue becomes unavailable, toxic, or credit-constrained.
- Real-time estimated total execution cost.

Core Reasoning Engine

Multi-objective optimization: For order size Q , side, urgency u , instrument i :

- For each eligible venue v , compute expected cost C_v , fill probability $P_{\text{fill},v}$, and credit cost.
- Objective: minimize $\sum w_v (C_v + \lambda(1-P_{\text{fill},v}))$ subject to credit limits and other constraints.
- Solve using linear programming or greedy allocation for low latency.

Decision Rules

- Small, non-urgent orders in liquid instruments → low-cost exchange with maker rebates, passive limit orders.
- Large block orders → dark pools first, then algorithmic slicing.
- FX spot → best streaming dealer with sufficient credit.
- Bonds/illiquid → RFQ to panel; select best quote after risk check.
- If venue toxicity > 0.8 → exclude temporarily.
- After hours: only venues with extended hours support.

Constraint Engine

- Regulatory: MiFID II best execution, Reg NMS; log venue selection rationale.
- Credit: pre-trade check; if trade would breach limit, exclude venue.
- Settlement risk: only allowed settlement methods (CLS, DVP).
- Venue membership and max order size enforced.

Validation Engine

- Backtest venue selection against historical order books.
- Compare actual execution costs with pre-trade estimates; recalibrate if bias.
- Monitor OTC RFQ performance (acceptance rate, price improvement).
- Weekly audit of credit checks.

Monitoring Rules

- Real-time venue latency, fill rate, rejection rate.
- If 5-min rejection rate >10%, flag as impaired.
- OTC dealer: if streaming quotes >100ms stale vs market, suppress use.
- Alert on venue outage or status change.

Learning & Adaptation

- Fill probability models for dark pools and RFQ: Bayesian posterior update.
- Cost model calibration: reinforcement learning (contextual bandits) to adjust venue weights online.
- Aggregated LOB prediction for depth beyond top of book.
- Venue performance decay tracking.

Failure Modes

Failure	Detection	Response
Primary exchange outage	Market data feed stops, order rejects	Route to alternative venues; halt if no liquidity.
OTC dealer credit breach	Real-time exposure exceeds limit	Suspend orders to that dealer; rebalance.
RFQ no-response	Timer expires with no quotes	Increase timeout or switch to streaming dealers.
Aggregated LOB inconsistencies due to latency	Timestamp comparison across venues	Temporarily ignore slow venue.
Regulatory violation detected	Post-trade analysis	Halt automated routing, human override.

Recovery Procedures

- After exchange outage: gradually increase allocation over 10 minutes after status returns to normal.
- After credit limit breach: apply for temporary increase or reduce positions; resume OTC routing after re-validation.
- After RFQ timeouts: remove dealer from panel for 30 minutes, then retry with smaller size.

Internal Memory

- Short-term (intraday): venue latency, fill probabilities, real-time exposure.
- Medium-term (monthly): historical performance metrics per instrument-venue pair.
- Long-term (annually): parameter estimates for cost models, regression coefficients for aggregated liquidity models.

Explainability Layer

- For each parent order, produce a “venue selection justification” showing top venues, scores, and exclusion reasons.
- Dashboard: average venue utilization, cost savings vs benchmark, credit usage headroom.

Health Metrics

- Venue routing effectiveness: % of orders executed on top-ranked venues.
- Execution cost relative to estimated: within $\pm 20\%$ for 90% of orders.
- Credit limit utilization: never exceeding 95%.
- SOR decision latency: p99 < 10 μ s for exchange routing, < 50 ms for RFQ initiation.
- RFQ hit rate: > 90% of RFQs receive at least one response.

Interfaces

- AITOS-EXEC-14
- AITOS-MICRO-08
- AITOS-RISK-10
- AITOS-EFF-07
- AITOS-SETTLE (future)
- AITOS-REGIME

Production-Grade Pseudocode

```
def route_order(instrument, side, size, urgency):
    venues = get_eligible_venues(instrument, side)
    venues = [v for v in venues if v.status == 'OPEN' and credit_check(v, size) and v.toxicity < 0.8]
    if not venues:
        return error("No eligible venue")
    cost_probs = []
    for v in venues:
        cost = estimate_total_cost(v, instrument, size, urgency)
        prob = estimate_fill_prob(v, instrument, size)
        cost_probs.append((v, cost, prob))
    sorted_venues = sorted(cost_probs, key=lambda x: x[1] + penalty*(1-x[2]))
    allocation = []
    total_size = size
    for v, cost, prob in sorted_venues:
        alloc = min(total_size, v.max_order_size) if prob > 0.5 else 0
        allocation.append((v, alloc))
        total_size -= alloc
        if total_size == 0: break
    if total_size > 0:
        allocation.append((primary_exchange, total_size))
    return allocation
```

Knowledge Graph



Machine-Readable JSON Schema

```
{
  "module_id": "AITOS-EXOTC-09",
  "version": "1.0",
  "state": {
    "venue_registry": "list[Venue]",
    "credit_limits": "dict[string, float]",
    "aggregated_lob": "dict[string, ConsolidatedLOB]"
  },
  "inputs": ["market_data_feeds", "order_requests", "credit_updates"],
  "outputs": ["venue_allocations", "liquidity_views", "alerts"],
}
```

```
"decision_engine": {  
  "algorithm": "multi-objective optimization",  
  "objective": "minimize expected execution cost subject to constraints",  
  "constraints": ["credit", "regulatory", "size_limits"]  
},  
"interfaces": ["AITOS-EXEC-14", "AITOS-MICRO-08", "AITOS-RISK-10"]  
}
```

Changelog

- v1.0: Initial production specification covering exchange and OTC venue management, SOR, RFQ, credit limits, and venue performance monitoring.

Module 04: Trading Session & Market Calendar Engine

(AITOS-SESSION-10)

Metadata

- Module ID: AITOS-SESSION-10
- Chapter: 10 – Trading Sessions
- Version: 1.0
- Last Updated: 2026-07-20
- Dependencies: AITOS-EXOTC-09, AITOS-MICRO-08, AITOS-EFF-07
- Status: Production

Purpose

To provide all other AITOS modules with a precise, real-time model of market session state for every traded instrument. The module encapsulates trading calendars, intraday volume/volatility curves, holiday schedules, and session-specific behaviors so that execution, alpha, and risk modules can adapt their strategies to the current market environment.

Scope

- Maintain a global calendar of trading sessions for all exchanges and OTC markets.
- Define standard session types: PRE-MARKET, REGULAR, AFTER-HOURS, CLOSED, AUCTION, MAINTENANCE.
- Provide a real-time session state with expected remaining time and transition probabilities.
- Provide intraday volume profiles normalized to ADV for each session phase.
- Model typical spread, depth, and volatility patterns as functions of session type and time within session.
- Support session-aware routing decisions, risk limits, and alpha decay.

Internal State Representation

Global Session Clock

```
{
  "timestamp": "2026-07-20T14:32:01.123Z",
  "active_sessions": {
    "US_EQUITIES": {
      "session_type": "REGULAR",
      "start_time": "09:30:00 EST",
      "end_time": "16:00:00 EST",
      "next_transition": "16:00:00 EST",
      "time_to_next": 5280.0
    }
  }
}
```

Per-Instrument Session Object

```
{
  "instrument_id": "AAPL",
  "market_venue": "NASDAQ",
  "session_type": "REGULAR",
  "sub_phase": "MID_DAY",
  "time_to_close_sec": 5280,
  "is_auction_period": false,
  "volume_profile_weight": 0.05,
  "typical_spread_multiplier": 1.0,
  "typical_impact_multiplier": 1.0,
  "holiday_risk_flag": false
}
```

Knowledge Representation

- Trading Calendar: deterministic schedule from exchange rules, with holiday exceptions. Represented as a decision tree.
- Volume Curve Model: piecewise linear or spline function $V(t)$ giving fraction of ADV traded up to time t .
- Session Transition Probability: small probability of unscheduled closures.
- Liquidity Parameter Curves: $\text{spread_factor}(t)$, $\text{depth_factor}(t)$, $\text{vol_factor}(t)$ as functions of normalized session time τ .

Inputs

- Venue status and schedule from AITOS-EXOTC-09.
- Real-time exchange notifications (halts, technical issues).
- Current UTC timestamp (atomic).
- Historical intraday volume data for calibration.
- Holiday/early-close calendars.

Outputs

- Continuous stream of session state updates.
- Session-based parameters: volume weight, spread multiplier, volatility multiplier.
- Time-to-event triggers (e.g., “closing auction in 5 minutes”).
- Risk module alerts when session is about to close and positions remain.
- Pre-market/post-market liquidity warnings.

Core Reasoning Engine

Finite-state machine (FSM) for each instrument, with deterministic scheduled transitions and probabilistic halts.

States: CLOSED, PRE_MARKET, REGULAR (with sub-states OPEN_AUCTION, CONTINUOUS, CLOSE_AUCTION), AFTER_HOURS, HALTED, CLOSED_EARLY.

Session-aware liquidity estimation:

$$\sigma(\tau) = \sigma_{\text{base}} \cdot g(\tau)$$

$$\text{impact_factor}(\tau) = h(\tau) \cdot (\text{volume_weight}(\tau))^{(-\gamma)}$$

Decision Rules

- Execution module: adapt order types per session (only limit orders in pre-/after-hours).
- Risk module: reduce position limits near close.
- Alpha: use session-specific alpha decay; first and last 30 minutes more reliable.
- Portfolio rebalancing: prefer close (MOC) orders.

Constraint Engine

- Must respect exchange-specific calendar nuances.
- Must not allow trading outside allowed times.
- Multi-asset session coordination for baskets.
- Circuit breaker rules: after Level 3 halt, all instruments to HALTED.

Validation Engine

- Daily consistency check of session state changes.
- Backtest volume curves; recalibrate if RMSE exceeds threshold.

- Simulate holiday impact.

Monitoring Rules

- Alert if session state change does not occur at scheduled time.
- Monitor spread/depth predictions against actual.
- Log every session transition.

Learning & Adaptation

- Volume curve update monthly.
- Spread/depth curve quarterly.
- Unscheduled halt model: update Poisson parameter.
- Holiday corrections annually.

Failure Modes

Failure	Detection	Response
Exchange calendar data error	Mismatch between state and actual	Flag emergency; pause trading for that instrument.
Incorrect volume profile	Large slippage relative to expectation	Recalibrate, reduce order sizes.
Delayed session opening	Market data not arriving	Extend pre-market; do not send orders until first trade.
Early unscheduled close	Exchange announcement	Cancel open orders, liquidate if risk mandates.

Recovery Procedures

- After delayed open: wait 2 minutes for order book stability.
- After erroneous holiday flag: manual override.
- After data gap: fall back to generic session curve.

Internal Memory

- Session state log.
- Calibration cache (volume/spread curves per instrument and session type).
- Holiday database for 5 years.

Explainability Layer

- Log session type, volume weight, spread multiplier for each trade.
- Dashboard: world clock with session indicators, color-coded by liquidity regime.

Health Metrics

- Session state accuracy: > 99.99% match to actual market status.
- Volume prediction error (MAPE) per 5-min bucket < 20%.
- Execution cost attributable to session mis-prediction < 1 bp.

Interfaces

- AITOS-EXEC-14
- AITOS-RISK-10
- AITOS-ALPHA
- AITOS-MICRO-08
- AITOS-PORT

Production-Grade Pseudocode

```
def update_session_state(instrument, current_time):
    calendar = get_calendar(instrument)
    session_times = calendar.get_sessions(current_time.date())
    if session_times is None:
        instrument.session_type = 'CLOSED'
        return
    for session in session_times:
        if session.start <= current_time < session.end:
            instrument.session_type = session.type
            instrument.time_to_next = (session.end - current_time).seconds
            tau = (current_time - session.start).total_seconds() / session.duration_seconds
            instrument.volume_weight = volume_curve(session.type, tau)
            instrument.spread_multiplier = spread_curve(session.type, tau)
            instrument.impact_multiplier = impact_curve(session.type, tau, instrument.volume_weight)
            return
    instrument.session_type = 'CLOSED'
```

Knowledge Graph

```
Trading Session Engine
├── SessionStateMachine
│   ├── States (CLOSED, PRE_MARKET, REGULAR, AFTER_HOURS, HALTED)
│   └── Transitions (scheduled, unscheduled)
├── SessionAwareLiquidity
│   ├── VolumeCurve
│   ├── SpreadCurve
│   └── ImpactCurve
├── CalendarDB
│   ├── ExchangeCalendars
│   └── HolidayExceptions
├── InterfaceContracts
│   ├── SessionStatePush
│   └── SessionParameterQuery
```

Machine-Readable JSON Schema

```
{
  "module_id": "AITOS-SESSION-10",
  "version": "1.0",
  "state": {
    "instrument_sessions": {
      "type": "dict",
      "value": {
        "session_type": "string",
        "tau": "float",
        "volume_weight": "float",
        "spread_multiplier": "float",
        "impact_multiplier": "float",
        "time_to_next_sec": "float",
        "is_auction": "bool"
      }
    },
    "global_calendar": "CalendarDB"
  },
  "inputs": ["current_time", "venue_status", "holiday_updates"],
  "outputs": ["session_state_stream", "session_parameters"],
  "decision_rules": {
    "execution": "adapt order type per session",
    "risk": "reduce limits near close"
  },
  "interfaces": ["AITOS-EXEC-14", "AITOS-RISK-10", "AITOS-ALPHA", "AITOS-MICRO-08"]
}
```

Changelog

- v1.0: Initial production specification. FSM with calendar, volume curves, session-aware liquidity estimation.

Module 05: Market Structure Engine (AITOS-MSTRUCT-11 v2.0)

Metadata

- Module ID: AITOS-MSTRUCT-11
- Chapter: 11 – Market Structure (Phase 2)
- Version: 2.0
- Last Updated: 2026-07-20
- Dependencies: AITOS-MICRO-08, AITOS-SESSION-10, AITOS-RISK-10
- Status: Production

Purpose

To provide the AI with a rigorous, probabilistic assessment of market structure states—trend, range, reversal, breakout—across multiple timeframes. The module translates raw price action into a real-time belief state over market regimes, driving strategy selection, position sizing, and risk management with quantified uncertainty.

Scope

- Detect swing points and classify trend/range using formal hypothesis tests.
- Maintain a Bayesian belief vector over structural states, updated on each bar close.
- Identify breakouts with a logistic model that fuses price, volume, and volatility.
- Model the regime-switching dynamics as a Markov chain for forward-looking state anticipation.
- Integrate with execution and alpha modules to bias order flow.
- Self-calibrate detection parameters online based on false-signal rate.

Internal State Representation

Per-Instrument-Timeframe State Object

```
{
  "instrument_id": "EUR/USD",
  "timeframe": "H1",
  "belief_state": {
    "prob_uptrend": 0.45,
    "prob_downtrend": 0.05,
    "prob_range": 0.40,
    "prob_breakout_upside": 0.07,
    "prob_breakout_downside": 0.03
  },
  "swing_points": [
    {"time": "2026-07-20T10:00", "type": "high", "price": 1.1050},
    {"time": "2026-07-20T09:00", "type": "low", "price": 1.0980}
  ],
  "key_levels": {
    "support": 1.0980,
    "resistance": 1.1050,
    "bos_level_up": null,
    "bos_level_down": 1.0980
  },
  "transition_matrix": {
    "uptrend_to_uptrend": 0.80
  },
  "detector_params": {
    "swing_window": 3,
    "bos_buffer_atr_mult": 0.2
  },
  "performance_metrics": {
    "false_breakout_rate_30d": 0.22,
    "trend_detection_lagBars": 2.1
  }
}
```



```
}
```

Knowledge Representation

- Regimes: {UPTREND, DOWNTREND, RANGE, BREAKOUT_UP, BREAKOUT_DOWN, INDETERMINATE}.
- Observation Model: For each bar, features x_t (HH count, HL count, distance from swing low/ATR, etc.) are computed. Likelihood $P(x_t | R)$ modeled via Gaussian mixtures per regime.
- Prior & Posterior Update: The prior belief is propagated through a first-order Markov transition matrix T (online estimate). Then posterior via Bayes: $b_t(R) \propto P(x_t | R) \cdot \tilde{b}_t(R)$.
- Breakout Model: Logistic regression on breakout features to compute probability.

Belief State

Vector b_t of regime probabilities. Entropy $H(b_t)$ used as confidence measure.

State Variables

As in the state object above.

Inputs

- OHLCV bars (all timeframes) from AITOS-MICRO-08.
- ATR(14) from Risk module.
- Volume profile (optional) from AITOS-VOL-16.
- Session liquidity score from AITOS-SESSION-10.

Outputs

- BeliefState broadcast to Alpha, Execution, Risk.
- Key level list (support, resistance, BOS triggers).
- Alerts: structural_shift_alert when probability of new regime exceeds 0.7.
- Strategy bias: allow_long, allow_short, prefer_mean_reversion derived from thresholds.

Core Reasoning Engine

1. Swing point detection ($O(1)$ per bar).
2. Feature extraction.
3. Bayesian belief update with exponential smoothing of sufficient statistics for likelihoods.
4. Breakout detection with logistic model.
5. BOS/CHoCH as secondary features.

Mathematical Models

- Markov transition: $\tilde{b}_t = T^T \cdot b_{t-1}$
- Posterior: $b_t(R) = P(x_t | R) \cdot \tilde{b}_t(R) / \Sigma$
- Breakout probability: $p = \sigma(w \cdot z + b)$

Decision Rules

- If $\text{prob_uptrend} > 0.6$ and $\text{prob_downtrend} < 0.1$: $\text{allow_long} = \text{True}$.
- If $\text{prob_range} > 0.5$: $\text{prefer_mean_reversion}$.
- If any breakout probability > 0.5 : trigger breakout strategy.

- If entropy > 1.5: reduce position size by 50%.

Constraint Engine

- Not operational during low-liquidity unless overridden.
- Respect tick size for key levels.
- For bars <1 min, skip Bayesian update; delegate to MICRO-08.
- Rate-limit updates to bar close.

Validation Engine

- Monthly backtest of regime labels vs manually annotated data (accuracy > 80%).
- False breakout rate < 0.3; recalibrate if higher.
- Reliability diagrams to check calibration.

Monitoring Rules

- Real-time entropy tracking.
- Alert if belief oscillates rapidly.
- Monitor transition matrix stability.
- Latency check: if update >1 ms per instrument, investigate.

Learning & Adaptation

- Online likelihood model update: exponential smoothing of mean/covariance.
- Transition matrix update: $T_{ij}(\text{new}) = (1-\beta) \cdot T_{ij}(\text{old}) + \beta \cdot I$
- Parameter tuning (swing window, BOS buffer) via grid search on validation set monthly.
- Breakout model retrained weekly.

Failure Modes

Failure	Detection	Response
Regime belief stuck	Entropy near 0 for long period	Inject random noise or reset model.
Frequent false breakouts	False breakout rate > 0.3	Increase threshold, retrain model.
Data gap	Gap detected	Discard updates during gap; reinitialize from current price structure.
Latency	Computation time >1 ms	Simplify features, offload to GPU.

Recovery Procedures

- Belief stuck: soft reset, widen prior, disable transition matrix for 5 bars.
- Model drift: emergency batch retrain over past 3 months.
- After market halt: use first 10 bars to re-establish swing points.

Internal Memory

- Ring buffer of last 200 bars, swing points, belief states.
- Sufficient statistics for online Gaussians.
- Transition count matrix (smoothed).
- Performance log of false breakout events.

Explainability Layer

- Log top 3 features contributing to regime change and likelihood ratios.
- Dashboard: price chart overlaid with colored regime shading.

Health Metrics

- Regime classification accuracy vs higher-TF consensus: > 80%.
- False breakout rate (30-day): < 25%.
- Belief calibration error (Brier score) < 0.05.
- Avg update latency per instrument: < 0.5 ms.

Interfaces

- AITOS-ALPHA
- AITOS-EXEC-14
- AITOS-RISK-10
- AITOS-SESSION-10
- AITOS-REGIME

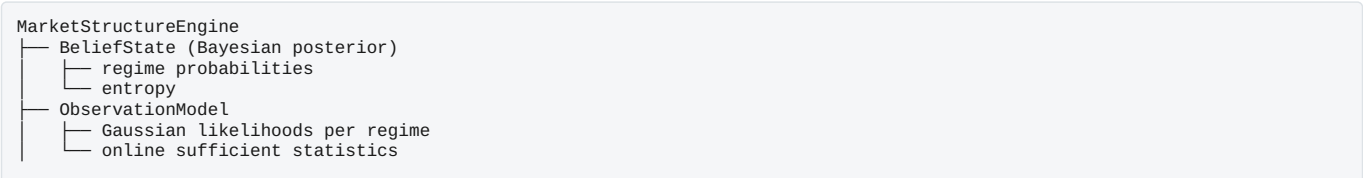
Production-Grade Pseudocode

```
class MarketStructureEngine:
    def __init__(self, instrument, timeframe):
        self.instr = instrument
        self.tf = timeframe
        self.belief = np.ones(6)/6
        self.transition = np.eye(6) * 0.9 + 0.1/6
        self.likelihood = GaussianLikelihood(num_regimes=6, feature_dim=6)
        self.swing_detector = SwingDetector(window=3)
        self.breakout_model = LogisticBreakoutModel()
        self.online_stats = OnlineSufficientStats(decay=0.05)

    def on_bar_close(self, bar):
        swings = self.swing_detector.update(bar)
        x = self.extract_features(swings, bar)
        pred_belief = self.transition.T @ self.belief
        log_likes = self.likelihood.log_prob(x)
        log_posterior = np.log(pred_belief + 1e-10) + log_likes
        log_posterior -= np.max(log_posterior)
        posterior = np.exp(log_posterior)
        self.belief = posterior / posterior.sum()
        dominant_regime = np.argmax(self.belief)
        self.online_stats.update(dominant_regime, x)
        if hasattr(self, 'prev_regime'):
            self.transition[self.prev_regime, dominant_regime] = 0.9 * self.transition[self.prev_regime, dominant_regime]
        self.prev_regime = dominant_regime
        brk_prob = self.breakout_model.predict(bar, self.key_levels)
        if brk_prob > 0.5:
            self.belief[REGIME.BREAKOUT_UP if bar.close > self.resistance else REGIME.BREAKOUT_DOWN] = brk_prob
            self.belief /= self.belief.sum()
            self.publish_belief()

    def extract_features(self, swings, bar):
        # returns vector of 6 features
        pass
```

Knowledge Graph



```
└─ TransitionMatrix (Markov)
└─ SwingDetector (deterministic)
└─ BreakoutDetector (logistic)
└─ KeyLevels (support/resistance)
└─ PerformanceMonitor
└─ Interfaces (Alpha, Execution, Risk, Session)
```

Machine-Readable JSON Schema

```
{
  "module_id": "AITOS-MSTRUCT-11",
  "version": "2.0",
  "state": {
    "instrument_states": {
      "type": "dict",
      "value": {
        "belief": "float[6]",
        "entropy": "float",
        "key_levels": {"support": "float", "resistance": "float"},
        "swings": "list[swing]",
        "transition": "float[6][6]",
        "perf": {"false_breakout_rate": "float"}
      }
    }
  },
  "inputs": ["ohlcv", "atr", "session_score"],
  "outputs": ["belief_state", "key_levels", "strategy_bias"],
  "decision_rules": "threshold on belief + entropy",
  "interfaces": ["AITOS-ALPHA", "AITOS-EXEC-14", "AITOS-RISK-10", "AITOS-SESSION-10"]
}
```

Changelog

- v2.0 (from v1.0): Converted from rule-based to full Bayesian framework with Markov transitions, online likelihood updates, deflated Sharpe validation, and self-calibration.

Module 06: Classical Technical Analysis Engine (AITOS-CTA-12)

Metadata

- Module ID: AITOS-CTA-12
- Chapter: 12 – Classical Technical Analysis
- Version: 1.0
- Last Updated: 2026-07-20
- Dependencies: AITOS-MICRO-08, AITOS-MSTRUCT-11, AITOS-SESSION-10, AITOS-RISK-10
- Status: Production

Purpose

To detect, classify, and assign probabilistic confidence to classical technical analysis patterns—candlestick formations, chart patterns, trendline breaks, and static support/resistance—augmenting the AI’s pattern recognition capabilities with well-established visual structures that human traders use. This module converts price and volume data into actionable pattern signals with quantified reliability.

Scope

- Candlestick patterns: Hammer, Shooting Star, Engulfing, Doji, Morning/Evening Star, Harami, etc.
- Chart patterns: Head and Shoulders, Double/Triple Top/Bottom, Triangles, Wedges, Flags, Pennants, Rectangle, Cup & Handle.
- Support & Resistance (static): Pivot points, historical swing levels, round numbers, volume-at-price nodes.
- Trendlines & Channels: Linear regression on swing points, breakouts.
- Dow Theory principles: Trend confirmation via higher-high/higher-low (overlap with MSTRUCT but handled as classical rule set), volume confirmation.
- Volume confirmation: Volume spike, trend of volume.

Internal State Representation

```
{
  "instrument_id": "AAPL",
  "timeframe": "D1",
  "active_patterns": [
    {
      "pattern_id": "HS_TOP_001",
      "type": "HEAD_AND_SHOULDERS_TOP",
      "status": "FORMING",
      "confidence": 0.72,
      "entry_level": 145.0,
      "target_level": 135.0,
      "stop_level": 150.0,
      "formation_start_time": "2026-06-10",
      "last_update": "2026-07-20",
      "volume_confirmation": 0.8,
      "quality_score": 0.65
    }
  ],
  "candlestick_signals": [
    {
      "time": "2026-07-20T09:30",
      "type": "BULLISH_ENGULFING",
      "confidence": 0.85,
      "context": "AT_SUPPORT"
    }
  ],
  "support_resistance_levels": {
    "static_support": [142.0, 140.5],
    "static_resistance": [148.0, 150.2],
    "pivot_points": {"R1": 149.0, "S1": 143.0}
  },
  "trendlines": [
    {
      "type": "uptrend",
      "start": "2026-06-01",
      "current_value": 144.8
    }
  ]
}
```

Knowledge Representation

- Pattern ontology: Boolean functions of OHLC for candlesticks; geometric templates on swing points for chart patterns.
- Bayesian confidence model: Prior based on historical hit rate; posterior updated with features (volume, trend alignment, symmetry). Logistic regression or Beta posterior.
- Pattern Lifecycle State Machine: IDLE → FORMING → CONFIRMED/INVALIDATED.

Belief State

For each pattern, probability of validity updated as new bars arrive.

State Variables

As in the internal state JSON.

Inputs

- OHLCV bars from AITOS-MICRO-08.
- Swing point list from AITOS-MSTRUCT-11.
- Market regime state from MSTRUCT.
- Session liquidity from AITOS-SESSION-10.

Outputs

- PatternSignal stream: type, direction, confidence, target, stop.
- Support/Resistance levels updated on each bar.
- Trendline break alerts.
- Quality scores for Alpha module.

Core Reasoning Engine

- Candlestick pattern recognition via rule-based classifier.
- Chart pattern detection using dynamic programming on swing points (template matching).
- Support/resistance from pivot points, historical clusters (KDE), round numbers.
- Trendline detection via robust linear regression on swing lows/highs; break when close beyond buffer.

Mathematical Models

- Bayesian pattern confidence: log-odds model $\log(P/(1-P)) = \beta_0 + \beta \cdot f$
- Pattern quality score: $Q = w_1 \cdot \text{Conf} + w_2 \cdot \text{VolConf} + w_3 \cdot \text{TrendAlign} + w_4 \cdot \text{Completeness}$

Decision Rules

- Chart pattern CONFIRMED + confidence >0.7 → initiate trade.
- Candlestick signals are confirmations only, aligned with regime.
- S/R levels fed to Execution for limit orders.
- Trendline breaks require volume > 1.2× avg and subsequent bar confirmation.

Constraint Engine

- Timeframe weighting (higher TFs more weight).
- Low-liquidity sessions: down-weight confidence.

- Adjust stops/targets away from round-number clusters.
- Max 5 active patterns per instrument.

Validation Engine

- Historical pattern database with outcomes; backtest hit rate, profit factor, MAE.
- Bayesian model calibration: reliability curve.
- Static S/R level bounce frequency.
- Walk-forward validation to avoid overfitting.

Monitoring Rules

- False breakout rate from trendlines/chart patterns; if >40%, increase thresholds.
- Detection latency <1 ms per bar.
- Alert on many patterns simultaneously.

Learning & Adaptation

- Online learning of pattern reliability: Beta posterior updated with each trade outcome.
- Parameter optimization quarterly via Bayesian optimization.
- Personalized pattern weighting: if pattern underperforms on an instrument, reduce weight.

Failure Modes

Failure	Response
False pattern detection due to noise	Increase smoothing; require more extreme values.
Overfitting	Regularization; raise min confidence threshold.
Misalignment of trendline due to outlier	Use robust regression (Huber); discard outliers.
Excessive computation	Optimize with vectorized ops; use GPU.

Recovery Procedures

- Invalidate pattern: cancel contingent orders; log for learning.
- Poor performance period: reduce capital allocation by 50% temporarily.
- Major regime change: reinitialize detection parameters.

Internal Memory

- Pattern history (last 500 instances with features/outcomes).
- Swing point cache (reused from MSTRUCT).
- Pivot point cache (daily).

Explainability Layer

- Textual justification: “Bullish Engulfing at support S1 (143.0) during uptrend, volume confirmation 1.8× avg, confidence 85%.”
- Visualization of patterns on chart.

Health Metrics

- Overall pattern hit rate (profit factor >1) > 0.55.
- False breakout rate < 0.35.
- Confidence calibration error (Brier score) < 0.05.
- Detection latency < 0.5 ms per bar.

Interfaces

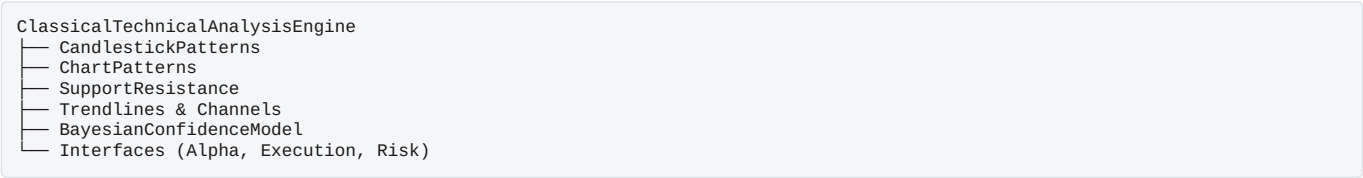
- AITOS-MSTRUCT-11
- AITOS-ALPHA
- AITOS-EXEC-14
- AITOS-RISK-10
- AITOS-SESSION-10
- AITOS-VOL-16 (future)

Production-Grade Pseudocode

```
class ClassicalTechnicalAnalysis:
    def __init__(self, instrument):
        self.instr = instrument
        self.patterns = []
        self.sr_levels = SREngine()
        self.trendline_mgr = TrendlineManager()
        self.candle_recognizer = CandlestickRecognizer()
        self.bayesian_model = BayesianPatternModel()

    def on_bar(self, bar, swings, regime):
        candles = self.candle_recognizer.detect(bar, self.prevBars)
        for c in candles:
            c.confidence = self.bayesian_model.get_confidence(c.type, c.features, regime)
            self.emit_signal(c)
        self.update_patterns(swings, bar.volume)
        self.sr_levels.update(bar, swings)
        self.trendline_mgr.update(swings, bar.close, bar.volume)
        self.update_beliefs()
```

Knowledge Graph



Machine-Readable JSON Schema

```
{
  "module_id": "AITOS-CTA-12",
  "version": "1.0",
  "state": {
    "active_patterns": "list[Pattern]",
    "sr_levels": {"support": "list[float]", "resistance": "list[float]"},
    "trendlines": "list[Trendline]",
    "candlestick_signals": "list[Signal]"
  },
  "inputs": ["OHLCV_bars", "swing_points", "regime_state", "session_liquidity"],
  "outputs": ["pattern_signals", "support_resistance_levels", "trendline_breaks"],
  "decision_rules": "use quality and confidence to size positions",
  "interfaces": ["AITOS-MSTRUCT-11", "AITOS-ALPHA", "AITOS-EXEC-14", "AITOS-RISK-10", "AITOS-SESSION-10"]
}
```

Changelog

- v1.0: Initial implementation of classical TA patterns with Bayesian confidence, lifecycle management, and integration with MSTRUCT.

Module 07: Wyckoff Method Engine (AITOS-WYCKOFF-13 v1.1)

Metadata

- Module ID: AITOS-WYCKOFF-13
- Chapter: 13 – Wyckoff Method
- Version: 1.1
- Last Updated: 2026-07-20
- Dependencies: AITOS-MSTRUCT-11, AITOS-CTA-12, AITOS-MICRO-08, AITOS-SESSION-10, AITOS-RISK-10
- Status: Production

Purpose

To detect and track Wyckoff accumulation and distribution schematics using a hierarchical probabilistic framework. This module infers the market’s current phase (Phase A–E) within a Wyckoff cycle, estimates the likelihood of future markup or markdown, and provides early entry/exit signals based on the interaction of price, volume, and time. It models the battle between composite operators (smart money) and the crowd, enabling the AI to anticipate institutional-grade turning points.

Scope

- Detection of Accumulation and Distribution patterns across all timeframes.
- Phase identification using a Hidden Semi-Markov Model (HSMM).
- Key events: SC, AR, ST, Spring/Upthrust, SOS, SOW, LPS, etc.
- Volume and price interaction analysis, including effort vs. result and absorption.
- Multi-timeframe Bayesian hierarchy.
- Confidence decay for inactive schematics.
- Composite Operator Score (COS) and Institutional Participation Index (IPI).
- Event reliability database with online Bayesian updating.
- Trading range quality scoring.
- Institutional liquidity sweep detection.
- Adaptive cause projection.
- Context integration for future Smart Money Concepts.

Internal State Representation

Per-Instrument Wyckoff Tracker (Extended)

```
{
  "instrument_id": "ES",
  "timeframe": "H4",
  "schematic_type": "ACCUMULATION",
  "current_phase": "C",
  "phase_probabilities": { "A": 0.05, "B": 0.10, "C": 0.60, "D": 0.20, "E": 0.05 },
  "phase_duration_estimates": {
    "A": { "mean": 5.2, "std": 2.1},
    "B": { "mean": 15.3, "std": 4.5},
    "C": { "mean": 3.1, "std": 1.0},
    "D": { "mean": 8.2, "std": 2.3},
    "E": { "mean": 12.0, "std": 3.8}
  },
  "key_levels": {
    "resistance": 4200.0, "support": 4100.0, "ice": 4150.0,
    "spring_low": 4080.0, "upthrust_high": 4220.0
  },
  "events": [
    { "event": "SC", "time": "2026-07-18", "price": 4095.0, "volume_ratio": 2.3},
    { "event": "AR", "time": "2026-07-19", "price": 4160.0, "volume_ratio": 1.1}
  ],
}
```

```
"cause_estimate": 120.0,  
"confidence": 0.75,  
"confidence_decay_factor": 0.02,  
"last_confirming_event_time": "2026-07-19T14:00",  
"composite_operator_score": 0.68,  
"institutional_participation_index": 0.72,  
"trading_range_quality_score": 0.82,  
"transition_probabilities": {  
  "A_to_B": 0.81, "B_to_C": 0.74, "C_to_D": 0.88, "D_to_E": 0.91  
},  
"event_reliability": {  
  "Spring": {"success_rate": 0.73, "sample_size": 420},  
  "Upthrust": {"success_rate": 0.68, "sample_size": 310},  
  "SOS": {"success_rate": 0.81, "sample_size": 315}  
}  
}
```

Knowledge Representation

- Wyckoff Schematic Models: canonical accumulation/distribution phases with duration distributions.
- Hidden Semi-Markov Model: hidden states with explicit duration modeling (Gamma/Weibull). Observations include price-volume features and microstructure data.
- Multi-Timeframe Hierarchy: higher TF beliefs serve as priors for lower TFs via weighted combination.
- Event Reliability Database: Beta-Binomial posteriors for each event type.
- Confidence Decay Model: exponential decay when no confirming events.
- Adaptive Cause Projection: dynamic multiplier based on asset class, ATR, volatility, COS, etc.

Belief State

- Phase probability vectors per timeframe.
- Schematic confidence (decayable).
- COS, IPI, range quality.
- Event reliability priors.

State Variables

As detailed in the internal state JSON.

Inputs

- OHLCV bars per timeframe from MICRO-08.
- Swing points from MSTRUCT-11.
- Microstructure data (OFI, iceberg flags, stop-hunt signals) from MICRO-EXT (future).
- Volume profile from VOL-16 (future).
- Session liquidity from SESSION-10.

Outputs

- Phase probabilities, COS, IPI.
- Signals (entry, target, stop) when Phase D confidence > threshold.
- Key levels and cause projections.
- Context variables for Context Engine.

Core Reasoning Engine

1. Trading range detection with quality scoring: range must have quality > threshold.
2. Wyckoff event detection: classifiers for Spring, Upthrust, SOS, etc., boosted by liquidity sweeps and event reliability priors.

3. HSMM forward update: uses Gaussian mixture emissions, directional transition constraints, duration distributions. Multi-timeframe prior injected as pseudo-observation.
4. Confidence decay: $C_t = C_{t-1} \cdot e^{(-\lambda \Delta t)}$, reset on confirming events.
5. COS computation: logistic ensemble of effort/result, absorption, OFI, iceberg flags, delta imbalance.
6. IPI: weighted combination of COS, liquidity sweeps, session liquidity.
7. Adaptive cause projection: learned model $k = f(\text{asset_class}, \dots)$ predicts expansion factor.

Mathematical Models

- HSMM forward recursion (directional): $\alpha_t(j) = \sum_d \alpha_{t-d}(i) \cdot a_{\{i,j\}} \cdot p_j(d) \cdot \Pi_s b_j(x_s)$
- Confidence decay: $C_t = C_{t-1} \cdot e^{(-\lambda \Delta t)} + \text{event_boost}$
- Event reliability: Beta posterior update.
- Range quality: $Q = \sum w_i \cdot q_i$
- Cause projection: $k = f(\text{features})$ (gradient boosting or neural net).

Decision Rules

- Signals only if schematic confidence >0.6, range quality >0.6, COS >0.5.
- Phase transition probability ($C \rightarrow D$) used to scale position size.
- If multi-TF hierarchy shows disagreement (e.g., Daily Accumulation vs H1 Distribution), reduce positions to 25% until alignment.
- If confidence decays below 0.3, exit positions and discard schematic.

Constraint Engine

- Min range quality threshold for Wyckoff analysis.
- Confidence decay may auto-remove inactive schematics.
- Multi-TF disagreement constraint.

Validation Engine

- Backtest HSMM vs HMM for phase classification accuracy.
- Validate confidence decay fit to typical inactivity periods.
- Adaptive cause projection: regression residuals uncorrelated.
- Monitor COS/IPI discrimination: AUC >0.65 for successful signals.

Monitoring Rules

- Track confidence decay curves; alert if many decay quickly.
- Monitor COS/IPI distribution across instruments; alert on widespread spikes.
- HSMM duration estimates updated daily; monitor for shifts.
- Event reliability database: flag if success rate drops below 0.5.

Learning & Adaptation

- Online HSMM parameter update via stochastic EM on sliding window.
- Adaptive cause model retrained monthly.
- COS weights adjusted via contextual bandit.
- Event reliability Beta parameters updated after every completed trade.

Failure Modes (Extended)

Failure	Response

HSMM computational lag	Fallback to HMM if latency >2 ms.
COS/IPI signals collapse (missing micro data)	Use classical Wyckoff rules without micro enhancement.
Multi-TF hierarchy over-regularizes	Reduce prior weight w temporarily when lower TF evidence accumulates.

Recovery Procedures

- If HSMM becomes unstable, revert to HMM with last known good parameters.
- If COS model fails, degrade gracefully with default COS of 0.5.
- Event reliability DB corruption: reload from backup; assume flat priors.

Internal Memory

- Circular buffer of HSMM alpha values.
- Duration distribution parameters per phase per instrument.
- Event reliability database (persisted).
- Recent COS and IPI values for trend analysis.

Explainability Layer

- Visualization includes confidence decay curve.
- For each signal, include COS and IPI values.
- Multi-timeframe dashboard with phase belief alignment indicator.

Health Metrics

Metric	KPI
HSMM update latency (with D_max=20)	< 1 ms per bar per instrument
Confidence decay fit R²	≥ 0.8
COS/IPI AUC for successful signals	≥ 0.65
Adaptive cause projection MAPE	< 15%

Interfaces

- AITOS-MSTRUCT-11
- AITOS-CTA-12
- AITOS-MICRO-EXT (future)
- AITOS-ALPHA
- AITOS-EXEC-14
- AITOS-RISK-10
- AITOS-CONTEXT (future)
- AITOS-SMC (future)

Production-Grade Pseudocode

```
class WyckoffEngineV1_1:
    def __init__(self, instrument, timeframes=['D','H4','H1','M15']):
        self.instr = instrument
        self.timeframes = timeframes
        self.hsms = {tf: HSMM(num_states=6) for tf in timeframes}
        self.event_clfs = {...}
        self.event_reliability = EventReliabilityDB()
        self.cause_model = AdaptiveCauseModel()
        self.cos_model = CompositeOperatorScoreModel()
        self.range_quality = RangeQualityScorer()
        self.beliefs = {tf: np.ones(6)/6 for tf in timeframes}
        self.confidences = {tf: 0.5 for tf in timeframes}
```

```

self.last_events = {tf: None for tf in timeframes}

def on_bar(self, tf, bar, swings, regime, micro_data=None):
    range_info = self.update_trading_range(tf, bar, swings)
    quality = self.range_quality.score(range_info, swings, bar)
    events = self.detect_events(bar, swings, micro_data)
    if events.empty():
        self.confidences[tf] *= exp(-self.decay_lambda[tf])
    else:
        self.confidences[tf] = min(1.0, self.confidences[tf] + 0.1)
        self.last_events[tf] = bar.index
    features = self.extract_features(bar, range_info, micro_data)
    prior = self.get_higher_tf_prior(tf)
    self.beliefs[tf] = self.hsmms[tf].forward_step(self.beliefs[tf], features, prior)
    cos = self.cos_model.compute(features, micro_data)
    ipi = self.compute_ipi(cos, events, micro_data)
    if (self.beliefs[tf][PHASE.D] > 0.7 and self.confidences[tf] > 0.6 and quality > 0.6):
        direction = 'LONG' if self.schematic_type[tf] == 'ACCUMULATION' else 'SHORT'
        target = self.cause_model.predict(range_info.height, features)
        stop = self.determine_stop(direction)
        self.emit_signal(direction, target, stop, self.beliefs[tf][PHASE.D],
                        cos=cos, ipi=ipi, transition_prob=self.transition_prob('C', 'D'))

```

Knowledge Graph

```

WyckoffEngine_v1.1
├── RangeQualityScorer
├── EventDetectors (extended with liquidity sweeps)
├── HSMM (with duration modeling)
├── MultiTimeframeBayesianHierarchy
├── ConfidenceDecayModel
├── CompositeOperatorScoreModel
├── InstitutionalParticipationIndex
├── EventReliabilityDB
├── AdaptiveCauseProjection
├── PhaseTransitionProbabilities
└── ContextIntegrationLayer

```

Machine-Readable JSON Schema (Extended)

```

{
  "module_id": "AITOS-WYCKOFF-13",
  "version": "1.1",
  "state": {
    "timeframes": {
      "type": "dict",
      "value": {
        "beliefs": "float[6]",
        "confidence": "float",
        "confidence_decay_lambda": "float",
        "last_event_index": "int",
        "range_quality": "float",
        "composite_operator_score": "float",
        "institutional_participation_index": "float",
        "phase_duration_estimates": "dict",
        "transition_probabilities": "dict"
      }
    },
    "event_reliability": "dict"
  },
  "inputs": ["OHLCV", "swings", "regime", "micro_data", "session_score"],
  "outputs": ["signals", "phase_beliefs", "context_vars", "cos", "ipi"],
  "decision_rules": "thresholds + multi-tf alignment",
  "interfaces": ["AITOS-MSTRUCT-11", "AITOS-CTA-12", "AITOS-MICRO-EXT", "AITOS-ALPHA", "AITOS-EXEC-14", "AITOS-RISK-10", "AITOS-SMC", "AITOS-CONTEXT"]
}

```

Changelog

- v1.0: Initial HSMM with classical Wyckoff phases, Bayesian phase tracking.
- v1.1: Added confidence decay, COS, IPI, multi-timeframe hierarchy, HSMM duration modeling, event reliability database, trading range quality,

institutional liquidity sweep detection, adaptive cause projection, and context integration layer.

End of reconstructed AITOS v1 specification.